Codebasics AI Engineering Bootcamp

Evaluation & Guardrail Pipeline Assignment

AI Evaluation & Guardrail Platform for Production AI Systems

Business Context

MediAssist Health Network now has multiple AI systems running across its hospitals and clinics, chat assistants like the one built for staff knowledge retrieval, plus several smaller pilots in scheduling and triage support. Leadership is no longer asking "can we build an AI system?" — every team has proven they can. The question now is: **"how do we know it's safe to keep running, and how do we know when it breaks?"**

Three incidents in the last quarter forced the issue:

- A pilot assistant confidently answered a dosage question using a hallucinated figure, an incident report was filed, and nobody could reconstruct what the model actually retrieved before answering.
- A support ticket claimed a chatbot leaked billing information to a non-billing role. The on-call engineer had no trace, no logs, and no way to confirm or rule this out for hours.
- A new model version was swapped in silently, answer quality dropped, and it took two weeks of user complaints before anyone noticed, there was no automated evaluation running in the background.

The Chief Risk Officer has commissioned a small AI platform team to stop firefighting incidents one at a time and instead build a **shared evaluation and guardrail layer** that any AI system in the network can be wired into before it goes live, and continuously afterward.

You are part of this platform team. Your job is to build an evaluation and guardrail pipeline: a system that sits around an existing AI application and continuously answers three questions — is this output safe to show a user, is this system behaving correctly right now, and is answer quality holding up over time.

Project Objective

Build an AI Evaluation & Guardrail Pipeline that wraps around a RAG or agentic chatbot (bring your own — an existing chatbot you've already built is the natural candidate to plug in) and provides:

- **Guardrails**: input and output safety checks on every request, applied before an unsafe prompt is processed and before an unsafe or leaking response reaches the user
- **Observability**: full tracing of every request through retrieval, reranking, and generation, with structured logs of every guardrail decision
- **AI Evaluation**: an automated, repeatable evaluation pipeline that scores retrieval and answer quality against a labeled question set

- **LLM-as-a-Judge**: a second model that grades each answer against a rubric and explains its reasoning
- **Heuristic Evals**: deterministic, rule-based checks that don't need an LLM to verify
- **Reporting**: a single evaluation report that turns all of the above into a pass/fail read on system health

Prerequisite: a working RAG or agentic chatbot to evaluate. If you already built one (for example, a retrieval assistant with role-based access control), use it as your target system. This pipeline is graded as its own system, the target chatbot is not re-graded here, it's the thing this pipeline watches.

Technical Requirements

01 Guardrail Layer (Input + Output)

An unsafe prompt or a leaking response must never reach the user, and the decision must be made by structured, checkable logic, not a hopeful prompt instruction.

REQUIREMENTS

- Build an **input guardrail** that runs before the target system processes a request: block prompt injection attempts, off-topic abuse, and requests that try to override role/access restrictions
- Build an **output guardrail** that runs after the target system responds, before the response reaches the user: check for leaked restricted content, PII, and unsafe or fabricated claims
- Use **OpenEvals** and/or **AWS Bedrock Guardrails** to implement at least one of the two layers, don't hand-roll every check from scratch
- Guardrail verdicts must be structured (e.g. JSON with a verdict and a reason), not inferred from prefix-matching a free-text response, and must **fail closed**: a malformed or missing verdict is treated as blocked, not passed
- Log the block reason internally for debugging, never echo it to the end user, show a generic, safe refusal instead

02 Observability & Tracing

If an answer goes wrong, you should be able to reconstruct exactly what happened, what was retrieved, what was reranked, what prompt reached the LLM, and what the guardrails decided, without asking the user to reproduce it.

REQUIREMENTS

- Instrument the full request path (retrieval → rerank → generation → guardrail checks) with **LangSmith** tracing, every request must produce a trace you can open and inspect end to end
- Every guardrail decision (allowed/blocked, and why) must be logged as a structured event, not just printed to console
- Capture latency and token usage per request, and expose them as basic metrics (a log line is acceptable, a dashboard is not required, but must be queryable)
- A reviewer must be able to pick any single logged request and answer "what did the system see, what did it decide, and why" using only the trace and logs, no re-running the request

03 AI Evaluation Pipeline

REQUIREMENTS

- Build a labeled evaluation set of at least 15 question/expected-answer pairs against your target system (cover both normal questions and at least a few adversarial/edge cases)
- Run **RAGAS** (or an equivalent RAG evaluation library) to compute, at minimum: **faithfulness**, **answer relevancy**, **context precision**, and **context recall**
- The evaluation must run as a repeatable script, not a one-off notebook cell, running it twice against an unchanged system should produce consistent results
- Report per-question scores and an aggregate score per metric

04 LLM-as-a-Judge

REQUIREMENTS

- Implement a judge step where a separate LLM call scores each answer against an explicit rubric (e.g. accuracy, completeness, appropriate refusal behavior, citation correctness)
- The judge must return a structured score plus a short written justification, not just a number
- State clearly in your README which model you used as the judge and why a separate model (or separate call) is used rather than having the system grade itself

05 Heuristic Evals

Not everything needs an LLM to check. Some things are just rules.

REQUIREMENTS

- Implement at least 4 deterministic, rule-based checks that require no LLM call, for example: every response includes at least one source citation, a restricted-role query about a restricted topic is refused (not just answered cautiously), response latency stays under a defined threshold, no response contains an empty or null answer field
- These checks must run as part of the same evaluation pipeline as Component 3, not as a separate, disconnected script

06 Evaluation Report

REQUIREMENTS

- Produce a single report (HTML, Markdown, or a simple dashboard) that consolidates: guardrail block/allow counts, RAGAS metric scores, LLM-as-a-judge scores, and heuristic eval pass/fail counts
- The report must state a clear overall verdict, e.g. a defined pass threshold per metric, and which specific checks failed if it doesn't pass
- Include at least one example each of: a guardrail correctly blocking an unsafe request, and a heuristic check correctly failing a bad response

Evaluation Criteria

Criterion	Weight
Guardrail layer blocks unsafe input and leaking output, verified with documented adversarial attempts, fails closed on malformed verdicts	25%
Observability: LangSmith tracing covers the full request path; guardrail decisions and metrics are logged as structured, queryable events	20%
AI Evaluation pipeline: RAGAS metrics computed correctly and repeatably against a real labeled question set	20%
LLM-as-a-Judge implemented with structured, justified scoring against an explicit rubric	15%
Heuristic Evals: at least 4 deterministic checks, integrated into the same pipeline	10%
Evaluation report clearly consolidates all signals into a pass/fail verdict	5%
Code quality, modularity, and README clarity	5%

Submission Instructions

1. Push your code to a **public GitHub repository**
2. Include a `README.md` with:
    - Setup instructions (API keys, how to run the guardrail layer and the evaluation pipeline)
    - Which target chatbot this pipeline is wired into (link the repo if it's a separate one)
    - At least 3 documented adversarial guardrail test cases, with the actual guardrail verdict shown
    - A sample evaluation report output
    - Any tool substitutions you made and why
3. Submit the repository link on the LMS

Tips

On Guardrails

Test with prompts that specifically try to make the guardrail itself fail open, malformed JSON, prompts that ask the model to "explain why you can't help" in a way that leaks the block reason, and confirm your fail-closed logic actually triggers.

On Observability

Don't just log successes. The traces that matter most are the ones for blocked or failed requests, that's where a reviewer will look first when something goes wrong.

On RAGAS

Faithfulness and answer relevancy will look fine on easy questions. Build a few genuinely hard or ambiguous questions into your eval set, that's where these metrics actually separate a good pipeline from a lucky one.

On LLM-as-a-Judge

Watch for the judge simply agreeing with confident-sounding answers. Include at least one deliberately wrong-but-confident answer in your test set to confirm the judge actually catches it.

On Heuristic Evals

These are your cheapest, fastest signal, run them first in your pipeline so a broken system fails fast before you spend LLM calls on RAGAS or the judge.